SOFTWARE ENGINEERING I · SPRINT 3

Static Analysis Report

Automated code quality and dependency security analysis of **FoodtruckApp**, a white-label pre-ordering web application, using the SonarQube JavaScript/TypeScript rule set and Trivy.

PREPARED BY
Unubileg

GITHUB
Unuvert1

DATE
October 20, 2026

COMMIT ANALYSED
5570aa3

0	0	0	PASS
CRITICAL VULNERABILITIES	BUGS	CODE VULNERABILITIES	QUALITY GATE

`https://github.com/Unuvert1/FoodtruckApp`

"This static analysis was generated using automated tools during this sprint."

1. Tools Used

Programming languages

The application is written in **TypeScript** throughout — 14,590 lines across 173 `.ts` and `.tsx` files — on Next.js 15 with React 19. The repository also contains SQL migration files generated by Prisma, and CSS. Runtime: Node.js 24.19.0, TypeScript compiler 5.9.3.

SonarQube analyser

Analyser	SonarJS 3.0.7 — SonarSource's official JavaScript/TypeScript analyser, the same rule engine SonarQube uses for this language
Rule profile	`recommended`, 269 rules available
Execution	Run locally through ESLint 9.39.5 rather than the SonarQube Server dashboard — the rules, RSPEC rule identifiers and issue classifications are SonarSource's own
Files analysed	170 source files

Trivy

Version	Trivy 0.76.0 (Aqua Security), official Windows x64 build
Scan type	**Filesystem scan** — `trivy fs` over the repository root
Scanners enabled	`vuln` (dependency vulnerabilities), `secret` (hard-coded credentials), `misconfig` (infrastructure misconfiguration)
Dependency source	the full npm dependency graph resolved from `package-lock.json`
Vulnerability database	downloaded fresh at scan time from the official Trivy database mirror

Where the scans were executed

Both scans were executed **locally**, on the developer's Windows 11 machine, against the working tree at commit `5570aa3`. Neither runs in CI yet; adding Trivy to a GitHub Actions workflow is the first item in the Sprint 4 plan in section 5.

> Scope note: the Sonar figures come from the SonarJS analyser run directly, so this report presents the rule-level results and evaluates the quality gate against SonarSource's published "Sonar way" conditions, rather than reproducing a hosted dashboard view.

2. Required Metrics

SonarQube

0	0	45	2
BUGS	VULNERABILITIES	CODE SMELLS	SECURITY HOTSPOTS

Metric	Value	Notes
Bugs	**0**	No reliability rule fired across 170 files.
Vulnerabilities	**0**	No injection, hard-coded-credential or weak-cryptography rule fired.
Code Smells	**45**	Maintainability issues across 5 rules. Full list in Appendix B.
Security Hotspots	**2**	Both reviewed and closed as not exploitable — see Appendix B.
Quality Gate	**PASS**	Evaluated against the "Sonar way" conditions; the condition table is in Appendix B.

Supporting measurements, each from a real run:

Measure	Value	Method
Tests passing	137 of 137	`vitest run`, 13 test files
Line coverage	81.29% (252/310)	`vitest --coverage` (v8), over the 17 `lib/` modules under test
Branch coverage	82.13% (262/319)	same run
Duplicated lines	0.27% (50 lines, 1 clone)	`jscpd` across 270 files

Trivy

0	8	6	0
CRITICAL	HIGH	MEDIUM	LOW

Severity	Count	Notes
CRITICAL	**0**	None.
HIGH	**8**	Highest CVSS 9.1 (postcss). All transitive.
MEDIUM	**6**	Includes 2 in Next.js itself, both fixed in 15.5.27.
LOW	**0**	None.
Total	**14**	Every finding is in a third-party package; none in first-party code.
Hard-coded secrets	**0**	Secret scanner enabled and clean.
Misconfigurations	**0**	No IaC or container config files exist in this repository to check.

3. Scope

What was scanned

- The entire application source tree — `app/`, `components/`, `lib/`, `middleware.ts` and the test files — 170 files.
- The complete resolved npm dependency graph from `package-lock.json`, production and development.
- The repository tree for hard-coded secrets and configuration misconfigurations.

What was excluded, and why

Excluded	Reason
`node_modules/`	Installed packages. Dependency vulnerabilities are covered through the lock file, which is the authoritative list.
`lib/generated/`	The Prisma client is generated code, recreated on every install and not written or reviewed by the team.
`.next/`, `out/`, `build/`	Build output.
`.env.local`	Local environment file. It is git-ignored and has never been committed, so it is not part of the codebase under review; excluding it also guarantees no credential is reproduced in this report.
`docs/`, `*.config.*`	Documentation and tool configuration, not application logic.

4. Trend

Baseline sprint — no prior comparison. This is the first sprint in which static analysis was run on FoodtruckApp, so there are no previous issue counts to compare against. These figures are the baseline Sprint 4 will be measured against:

Measure	Sprint 3 baseline	Sprint 4 target
Trivy HIGH	8	0–2 (only those with no upstream fix)
Trivy MEDIUM	6	2
Code Smells	45	under 25
Bugs / Vulnerabilities	0 / 0	hold at 0 / 0
Functions over complexity 15	5	0

5. Reflection

The most problematic area is the **third-party dependency chain** rather than the team's own code: all 14 vulnerabilities, including every one of the 8 HIGH findings, come from transitive packages, and two thirds trace back to just two entries in `package.json` — the `shadcn` CLI, declared as a runtime dependency although it is only ever used to scaffold components, and Next.js, which pins an outdated `postcss` internally. To reduce this next sprint we will move `shadcn` into `devDependencies` and upgrade Next.js to 15.5.27 — together these clear 10 of the 14 findings — then add a Trivy step to CI so a vulnerable dependency fails the build instead of waiting for an end-of-sprint scan. On the code-quality side the project is clean for reliability and security (0 bugs, 0 vulnerabilities), so the remaining work is maintainability: splitting the five functions that exceed the cognitive complexity limit and replacing the 24 nested ternary expressions, which are the single largest source of code smells.

“This static analysis was generated using automated tools during this sprint.”

Appendix A — Trivy Detailed Results

Tool	Trivy 0.76.0 (Aqua Security)
Scan type	Filesystem — `trivy fs`
Target	Repository root at commit `5570aa3`
Scan date	October 20, 2026

Command

```
trivy fs --scanners vuln,secret,misconfig \
  --skip-dirs node_modules --skip-dirs .next --skip-dirs .git \
  --skip-dirs lib/generated --skip-files .env.local \
  --format json -o trivy.json .
```

A second run with `--include-dev-deps` returned the identical 14 findings, confirming no vulnerability is hidden in the development-only dependency tree.

All findings

Severity	Vulnerability	Package	Installed	Fixed in	CVSS
HIGH	CVE-2026-104850	@modelcontextprotocol/sdk	1.30.0	1.31.0	7.5
HIGH	CVE-2026-93687	braces	3.0.3	no fix yet	7.5
HIGH	CVE-2026-40345	deepmerge-ts	7.1.5	8.0.0	—
HIGH	GHSA-3f6p-5ww8-9rcr	mysql2	3.15.3	3.22.0	—
HIGH	CVE-2026-45623	postcss	8.4.31	8.5.12	9.1
HIGH	CVE-2026-73646	postcss	8.4.31	8.5.18	7.5
HIGH	GHSA-wq5f-xc86-pv6w	sharp	0.35.4	0.35.5	—
HIGH	CVE-2026-93749	source-map-js	1.2.1	1.2.2	7.5
MEDIUM	CVE-2026-111757	hono	4.13.8	4.13.11	5.3
MEDIUM	GHSA-rgwj-5xj2-c3m3	mysql2	3.15.3	3.23.1	5.9
MEDIUM	CVE-2026-94484	next	15.5.25	16.3.8, 15.5.27	4.8
MEDIUM	CVE-2026-94543	next	15.5.25	15.5.27, 16.3.8	5.3
MEDIUM	CVE-2026-41305	postcss	8.4.31	8.5.10	6.1
MEDIUM	CVE-2026-69153	postcss	8.4.31	8.5.23	5.3

What each finding is, and where it comes from

Severity	Vulnerability	Description	Dependency path
HIGH	CVE-2026-104850	MCP TypeScript SDK: Information disclosure via unbound OAuth credentials	`shadcn 4.21.0` → `@modelcontextprotocol/sdk`

HIGH	CVE-2026-93687	Denial of Service via Stack Overflow from Deeply Nested Patterns	`shadcn 4.21.0 → fast-glob → micromatch → braces`
HIGH	CVE-2026-40345	DeepmergeTS has stack exhaustion when merging recursive object graphs	`prisma 7.10.0 → @prisma/config → deepmerge-ts`
HIGH	GHSA-3f6p-5ww8-9rcr	Auth Plugin Downgrade to mysql_clear_password Leaks Plaintext Credentials	`prisma 7.10.0 → mysql2`
HIGH	CVE-2026-45623	Information disclosure and denial of service via crafted CSS input	`next 15.5.25 → postcss 8.4.31 (pinned by Next)`
HIGH	CVE-2026-73646	Information disclosure via path traversal in source map auto-loading	`next 15.5.25 → postcss 8.4.31 (pinned by Next)`
HIGH	GHSA-wq5f-xc86-pv6w	Vulnerability in librsvg dependency CVE-2026-96889	`next 15.5.25 → sharp`
HIGH	CVE-2026-93749	Denial of Service via malformed indexed source maps	`next 15.5.25 → postcss → source-map-js`
MEDIUM	CVE-2026-111757	Information disclosure via malformed percent-encoded paths in serveStatic	`shadcn 4.21.0 → @modelcontextprotocol/sdk → hono`
MEDIUM	GHSA-rgwj-5xj2-c3m3	Unbounded zlib inflate in compressed MySQL protocol handler allows decompression-bomb DoS	`prisma 7.10.0 → mysql2`
MEDIUM	CVE-2026-94484	Denial of Service via response cache poisoning	`direct dependency`
MEDIUM	CVE-2026-94543	Response cache poisoning via improper route binding	`direct dependency`
MEDIUM	CVE-2026-41305	Cross-Site Scripting (XSS) via improper escaping of style closing tags	`next 15.5.25 → postcss 8.4.31 (pinned by Next)`
MEDIUM	CVE-2026-69153	Information disclosure via crafted sourceMappingURL	`next 15.5.25 → postcss 8.4.31 (pinned by Next)`

Where the risk concentrates

Root entry in package.json	Findings	HIGH	MEDIUM	Assessment
`shadcn 4.21.0`	3	2	1	A scaffolding CLI declared under `dependencies` rather than `devDependencies`. It pulls in an MCP server SDK and the `hono` web framework, neither of which this application ever runs. Nothing imports it at runtime.
`next 15.5.25`	8	4	4	Two findings are in Next.js itself (cache poisoning, both fixed in 15.5.27); the remainder are the older `postcss` 8.4.31 that Next pins internally, plus `sharp`.
`prisma 7.10.0`	3	2	1	The two `mysql2` findings are in a MySQL driver bundled with the Prisma CLI. This project uses PostgreSQL, so that driver is never loaded and neither issue is reachable.

Remediation plan

Priority	Action	Clears

1	Move `shadcn` out of `dependencies`; invoke it as `npx shadcn@latest` when scaffolding.	3 findings (2 HIGH, 1 MEDIUM)
2	Upgrade Next.js to 15.5.27.	2 MEDIUM directly, and refreshes the pinned `postcss` and `sharp` behind 7 more
3	Add `trivy fs --exit-code 1 --severity HIGH,CRITICAL` to a GitHub Actions workflow.	Prevents the next one arriving unnoticed
4	Accept and document the remainder: `mysql2` (unreachable — wrong database driver) and `braces` (no upstream fix published).	—

These changes were deliberately **not** applied during this sprint: the scan ran after the sprint's feature work was merged, and a dependency upgrade on the final day would have shipped untested. They are the first items for Sprint 4.

Appendix B — SonarQube Detailed Results

Analyser	SonarJS 3.0.7 (SonarSource)
Runner	ESLint 9.39.5, flat config, 269 rules available
Files analysed	170 (14,590 lines of TypeScript)
Target	Repository root at commit `5570aa3`

Command

```
npm install --no-save eslint-plugin-sonarjs@^3
npx eslint --config eslint.sonar.config.mjs --no-config-lookup \
  -f json -o sonarjs.json .
```

47 issues in total, raised by 6 distinct rules, in 30 of the 170 files analysed.

Issues by rule

RSPEC	Rule	Type	Count	Description
S3358	`no-nested-conditional`	Code Smell	**24**	Ternary operators should not be nested
S1135	`todo-tag`	Code Smell	**8**	Track uses of "TODO" tags
S3776	`cognitive-complexity`	Code Smell	**5**	Cognitive Complexity of functions should not be too high
S4624	`no-nested-template-literals`	Code Smell	**4**	Template literals should not be nested
S3735	`void-use`	Code Smell	**4**	"void" should not be used
S5852	`slow-regex`	**Hotspot**	**2**	Using slow regular expressions is security-sensitive

> Issue types follow each rule's published RSPEC classification. S5852 is defined as “security-sensitive”, which is a Security Hotspot — an item requiring human review rather than a confirmed defect. The other five are maintainability rules.

Security hotspot review

A Security Hotspot requires a developer to review the surrounding context and record a verdict. Both were reviewed by tracing the data that reaches each regular expression.

`components/dashboard/onboarding-form.tsx:25`

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

```
.replace(/[^a-z0-9]+/g, "-")
.replace(/^-+|-+$/g, "")
.slice(0, 40);
```

Verdict: not exploitable. This is `slugify` inside `onboarding-form.tsx`, a `"use client"` component: the expression runs in the visitor's own browser, on the truck name they are typing into their own form, and the result is truncated to 40 characters. `/-+$/` can degrade to quadratic time on a long run of dashes, but the only machine affected is the typist's own browser tab. No server-side request evaluates this expression, so there is no denial-of-service surface.

`lib/money.ts:71`

Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

```
const fraction = String(bps % 100).padStart(2, "0").replace(/0+$/, "");
```

Verdict: not exploitable. `bps % 100` is an integer between 0 and 99, so after `padStart(2)` the subject string is always exactly two characters. `/0+$/` cannot backtrack meaningfully over two characters, and the input is a computed value rather than user-supplied data.

Both hotspots are **reviewed and accepted**, satisfying the "security hotspots reviewed" gate condition. They were left unchanged rather than rewritten, since rewriting code with no reachable risk is churn; if either expression moves to the server, the verdict must be revisited.

Functions above the cognitive complexity limit

The limit is 15. Five functions exceed it, and the two worst are both screens written during this sprint.

File	Line	Complexity	Over by
`components/dashboard/pos/pos-screen.tsx`	41	**27**	+12
`components/dashboard/schedule/stop-sheet.tsx`	44	**23**	+8
`lib/schedule/saveService.ts`	50	**22**	+7
`components/storefront/menu-list.tsx`	249	**17**	+2
`lib/pricing.ts`	80	**17**	+2

Coverage and duplication

Measure	Value	Scope
Tests	137 passing / 137	13 test files
Lines	81.29% (252/310)	The 17 `lib/` modules the tests import. The v8 provider instruments only files a test loads, so this figure covers the business-logic layer; the `app/` and `components/` layers have no automated tests yet.
Branches	82.13% (262/319)	
Functions	82.02% (73/89)	
Duplicated lines	0.27% (50 lines)	One clone of 51 lines: the modifier-group picker exists twice, in `components/storefront/item-sheet.tsx` and `components/dashboard/pos/pos-options.tsx`.

Quality gate

Condition	Threshold	Measured	Result
Reliability rating	A	A — 0 bugs	PASS
Security rating	A	A — 0 vulnerabilities	PASS
Security hotspots reviewed	100%	100% — 2 of 2	PASS

Coverage	≥ 80%	81.29% on the modules under test	PASS
Duplicated lines	≤ 3%	0.27%	PASS

Overall result: PASS against the "Sonar way" conditions. The coverage condition is met over the business-logic modules that carry automated tests; extending test coverage to the UI layer is tracked as Sprint 4 work.

Complete issue list

#	File	Line	Rule	Message
1	`app/(platform)/invite/[token]/page.tsx`	25	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
2	`app/(storefront)/[truckSlug]/page.tsx`	54	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
3	`components/dashboard/item-form-sheet.tsx`	152	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
4	`components/dashboard/order-detail-actions.tsx`	44	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
5	`components/dashboard/place-combobox.tsx`	187	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
6	`components/dashboard/place-combobox.tsx`	250	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
7	`components/dashboard/pos/pos-options.tsx`	138	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
8	`components/dashboard/pos/pos-screen.tsx`	97	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
9	`components/dashboard/pos/pos-screen.tsx`	99	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
10	`components/dashboard/pos/pos-screen.tsx`	121	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
11	`components/dashboard/pos/pos-screen.tsx`	182	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.
12	`components/dashboard/schedule/spot-sheet.tsx`	92	`no-nested-conditional`	Extract this nested ternary operation into an independent statement.

13	components/dashboard/schedule/stop-sheet.tsx	122	no-nested-conditional	Extract this nested ternary operation into an independent statement.
14	components/dashboard/schedule/stop-sheet.tsx	135	no-nested-conditional	Extract this nested ternary operation into an independent statement.
15	components/dashboard/schedule/stop-sheet.tsx	138	no-nested-conditional	Extract this nested ternary operation into an independent statement.
16	components/dashboard/schedule/stop-sheet.tsx	230	no-nested-conditional	Extract this nested ternary operation into an independent statement.
17	components/dashboard/settings/notifications-form.tsx	112	no-nested-conditional	Extract this nested ternary operation into an independent statement.
18	components/dashboard/settings/team-list.tsx	262	no-nested-conditional	Extract this nested ternary operation into an independent statement.
19	components/storefront/item-sheet.tsx	218	no-nested-conditional	Extract this nested ternary operation into an independent statement.
20	components/storefront/menu-list.tsx	161	no-nested-conditional	Extract this nested ternary operation into an independent statement.
21	components/storefront/menu-list.tsx	302	no-nested-conditional	Extract this nested ternary operation into an independent statement.
22	components/storefront/order-status.tsx	47	no-nested-conditional	Extract this nested ternary operation into an independent statement.
23	lib/schedule/ordering-window.ts	27	no-nested-conditional	Extract this nested ternary operation into an independent statement.
24	lib/schedule/saveService.ts	86	no-nested-conditional	Extract this nested ternary operation into an independent statement.
25	app/(dashboard)/dashboard/actions.ts	63	todo-tag	Complete the task associated to this "TODO" comment.
26	app/(storefront)/[truckSlug]/checkout/actions.ts	47	todo-tag	Complete the task associated to this "TODO" comment.
27	components/dashboard/order-detail-actions.tsx	250	todo-tag	Complete the task associated to this "TODO" comment.
28	components/dashboard/settings/payments-status.tsx	21	todo-tag	Complete the task associated to this "TODO" comment.

29	lib/orders/markOrderPaid.ts	3	todo-tag	Complete the task associated to this "TODO" comment.
30	lib/tenant.ts	535	todo-tag	Complete the task associated to this "TODO" comment.
31	lib/tenant.ts	879	todo-tag	Complete the task associated to this "TODO" comment.
32	middleware.ts	4	todo-tag	Complete the task associated to this "TODO" comment.
33	components/dashboard/pos/pos-screen.tsx	41	cognitive-complexity	Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed.
34	components/dashboard/schedule/stop-sheet.tsx	44	cognitive-complexity	Refactor this function to reduce its Cognitive Complexity from 23 to the 15 allowed.
35	components/storefront/menu-list.tsx	249	cognitive-complexity	Refactor this function to reduce its Cognitive Complexity from 17 to the 15 allowed.
36	lib/pricing.ts	80	cognitive-complexity	Refactor this function to reduce its Cognitive Complexity from 17 to the 15 allowed.
37	lib/schedule/saveService.ts	50	cognitive-complexity	Refactor this function to reduce its Cognitive Complexity from 22 to the 15 allowed.
38	app/(platform)/invite/[token]/actions.ts	17	no-nested-template-literals	Refactor this code to not use nested template literals.
39	app/(platform)/invite/[token]/page.tsx	38	no-nested-template-literals	Refactor this code to not use nested template literals.
40	components/dashboard/order-detail-actions.tsx	106	no-nested-template-literals	Refactor this code to not use nested template literals.
41	lib/email.ts	32	no-nested-template-literals	Refactor this code to not use nested template literals.
42	components/dashboard/new-order-chime.tsx	45	void-use	Remove this use of the "void" operator.
43	components/dashboard/new-order-chime.tsx	53	void-use	Remove this use of the "void" operator.
44	lib/saved-customer.test.ts	13	void-use	Remove this use of the "void" operator.
45	lib/saved-customer.test.ts	14	void-use	Remove this use of the "void" operator.

46	components/dashboard/onboarding-form.tsx	25	slow-regex	Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.
47	lib/money.ts	71	slow-regex	Make sure the regex used here, which is vulnerable to super-linear runtime due to backtracking, cannot lead to denial of service.

“This static analysis was generated using automated tools during this sprint.”